



PROCESO DE GESTIÓN DE FORMACIÓN PROFESIONAL INTEGRAL FORMATO GUÍA DE APRENDIZAJE

1) IDENTIFICACIÓN DE LA GUIA DE APRENDIZAJE

- **Denominación del Programa de Formación:** Tecnología en Análisis y Desarrollo de Software
- **Código del Programa de Formación:** 3065369
- **Nombre del Proyecto:** Construcción de software integrador de tecnologías orientadas a servicios
- **Fase del Proyecto:** Ejecución
- **Actividad de Proyecto:** Introducción a funciones
- **Competencia:** DESARROLLAR LA SOLUCIÓN DE SOFTWARE DE ACUERDO CON EL DISEÑO Y METODOLOGÍAS DE DESARROLLO
- **Resultados de Aprendizaje Alcanzar:** Resolver procesos lógicos a través de la implementación de Algoritmos y el lenguaje de programación seleccionado.
- **Duración de la Guía:** 8 Horas 30 minutos

2) PRESENTACIÓN

Aprendiz SENA:

En el desarrollo de aplicaciones web modernas, la interacción dinámica entre el usuario y la interfaz es un elemento fundamental para garantizar experiencias intuitivas, eficientes y funcionales. En este contexto, el **Modelo de Objetos del Documento (DOM – Document Object Model)** se constituye como un componente clave que permite a los desarrolladores acceder, manipular y actualizar de forma dinámica los elementos de una página web utilizando lenguajes de programación como JavaScript.

La presente **guía de aprendizaje** tiene como propósito orientar al aprendiz en la comprensión y aplicación del DOM, facilitando el reconocimiento de su estructura, funcionamiento y uso dentro del desarrollo frontend. A través de actividades prácticas y situaciones de aprendizaje contextualizadas, el aprendiz podrá fortalecer sus



competencias técnicas, mejorando la interacción con documentos HTML y optimizando la respuesta de las aplicaciones web ante eventos del usuario.

Al finalizar esta guía, el aprendiz estará en capacidad de comprender el rol del DOM en el desarrollo web y aplicar sus conceptos fundamentales para manipular elementos, responder a eventos y mejorar la funcionalidad de interfaces interactivas.

3) FORMULACIÓN DE LAS ACTIVIDADES DE APRENDIZAJE

A. Actividades de reflexión inicial.

En el proceso de formación en desarrollo de software, los aprendices han tenido la oportunidad de trabajar con **HTML y CSS**, creando estructuras y estilos para páginas web. A partir de estas experiencias, es posible reconocer que una página web no es solo un conjunto de etiquetas y estilos, sino un entorno vivo que responde a las acciones del usuario, como hacer clic en un botón, completar un formulario o interactuar con un menú.

Sin embargo, surge una pregunta clave: **¿cómo es posible que una página web cambie su contenido, estructura o comportamiento sin necesidad de recargarla?** Esta inquietud marca el punto de partida para el nuevo conocimiento que se abordará en esta guía.

El **Modelo de Objetos del Documento (DOM)** permite comprender cómo el navegador interpreta un documento HTML y lo transforma en una estructura jerárquica de objetos que puede ser manipulada mediante JavaScript. A través del DOM, los desarrolladores pueden acceder a los elementos de una página, modificar su contenido, cambiar estilos, gestionar eventos y crear interfaces dinámicas que mejoran la experiencia del usuario.

Esta reflexión inicial invita al aprendiz a reconocer la relación entre los conocimientos adquiridos previamente y el nuevo aprendizaje que está por desarrollarse. Al finalizar esta guía, el aprendiz no solo comprenderá qué es el DOM, sino que será capaz de utilizarlo de forma práctica para transformar páginas web estáticas en aplicaciones interactivas,



fortaleciendo así sus competencias técnicas y su proyección en el ámbito del desarrollo web.

Actividad

Imagina que has desarrollado una página web utilizando únicamente HTML y CSS. La página muestra información correctamente, tiene un diseño atractivo y se visualiza bien en diferentes dispositivos. Sin embargo, el cliente solicita las siguientes mejoras:

- Mostrar u ocultar información cuando el usuario hace clic en un botón.
- Cambiar el texto de un elemento sin recargar la página.
- Validar un formulario antes de enviarlo.
- Responder de forma inmediata a las acciones del usuario.

Ante esta solicitud, surgen varias dificultades cuando no se cuenta con un mecanismo de interacción dinámica.

Desarrollo de la actividad

1. Análisis individual

Responde las siguientes preguntas en tu cuaderno o archivo digital:

- ¿Qué limitaciones tiene una página web construida solo con HTML y CSS?
- ¿Qué sucede si el usuario necesita interactuar con la página y recibir una respuesta inmediata?
- ¿Es posible modificar el contenido de una página sin recargarla usando únicamente HTML y CSS? Justifica tu respuesta.

2. Observación práctica

El instructor mostrará o el aprendiz revisará un ejemplo sencillo de una página web estática (HTML + CSS) que contenga:

- Un botón
- Un texto informativo



- Un formulario básico

A partir del ejemplo, reflexiona:

- ¿Qué acciones del usuario no generan ningún cambio en la página?
- ¿Qué experiencias pueden resultar frustrantes para el usuario final?

3. Reflexión guiada

Completa la siguiente afirmación:

“Antes de trabajar con mecanismos como el DOM, el principal problema del desarrollo web era que _____.”

Estrategias didácticas activas:

- Activación de saberes previos
- Aprendizaje basado en problemas (ABP)
- Observación guiada
- Preguntas reflexivas.
- Aprendizaje colaborativo

Duración de la actividad 30 minutos.

B. Actividades de contextualización e identificación de conocimientos necesarios para el aprendizaje.

Hasta ahora has identificado que una página web creada solo con HTML y CSS tiene limitaciones cuando se trata de interactuar con el usuario. En este momento del proceso formativo, es importante comprender qué sucede “detrás de cámaras” cuando el navegador carga una página web y cómo es posible interactuar con sus elementos de manera dinámica.

Cuando el navegador interpreta un documento HTML, no lo maneja como un simple archivo de texto. En su lugar, lo convierte en una estructura organizada en forma de árbol,



donde cada etiqueta se transforma en un objeto con propiedades y comportamientos. A esta estructura se le conoce como **Modelo de Objetos del Documento (DOM)**.

Gracias al DOM, es posible que un lenguaje como **JavaScript** acceda a los elementos de la página, los modifique, los oculte, los muestre o responda a las acciones del usuario, como clics, escritura en formularios o movimientos del mouse. Esto es lo que permite que una página web deje de ser estática y se convierta en una aplicación interactiva.

En este momento aprenderás a reconocer qué es el DOM, cómo se relaciona con el HTML y por qué es una pieza fundamental en el desarrollo web moderno. Esta comprensión te permitirá avanzar hacia la creación de interfaces más dinámicas, funcionales y alineadas con las necesidades reales de los usuarios.

¿Qué es el DOM?

Cuando tú escribes un archivo **HTML**, en realidad estás creando una estructura con etiquetas como `<html>`, `<head>`, `<body>`, `<div>`, `<p>`, entre otras. Sin embargo, el navegador **no trabaja directamente con el archivo HTML tal como lo escribes**.

Cuando la página se carga, el navegador hace lo siguiente:

1. Lee el archivo HTML.
2. Interpreta cada etiqueta.
3. Convierte esas etiquetas en una estructura organizada de objetos.

A esta estructura se le llama **DOM (Document Object Model)** o **Modelo de Objetos del Documento**.

El DOM como un árbol

Para entender mejor el DOM, imagina un **árbol**:

- El documento completo es la **raíz** del árbol.
- Cada etiqueta HTML es una **rama** o **nodo**.
- Las etiquetas que están dentro de otras etiquetas son **ramas hijas**.



Ejemplo sencillo:

```
<body>
  <h1>Bienvenido</h1>
  <p>Este es un ejemplo</p>
</body>
```

El navegador lo interpreta así:

- body es el nodo padre.
- h1 y p son nodos hijos.
- El texto dentro de las etiquetas también es un nodo.

Esto significa que **todo en la página es un objeto**: etiquetas, textos, atributos, imágenes y botones.

¿Por qué el DOMO es tan importante?

Antes de conocer el DOM, una página web:

- Mostraba información, pero no reaccionaba.
- No podía cambiar su contenido sin recargarse.
- No respondía a las acciones del usuario.

Gracias al DOM, ahora es posible:

- Cambiar textos en tiempo real.
- Mostrar u ocultar elementos.
- Modificar estilos desde JavaScript.
- Responder a eventos como clics, teclas o formularios.

En pocas palabras:

El DOM es lo que permite que una página web “**cobre vida**”

Ejemplo

Piensa en un botón que dice “**Mostrar mensaje**”.



- Sin DOM: el botón está ahí, pero no hace nada.
- Con DOM: al hacer clic, aparece un mensaje en pantalla.

Esto sucede porque JavaScript **accede al botón a través del DOM** y le indica qué debe hacer cuando el usuario interactúa con él.

Partes del DOM

Para poder trabajar con el DOM, primero necesitas conocer sus **elementos principales**. No es memorizar definiciones, es entender **qué representa cada parte y para qué sirve**.

1. El objeto document

El DOM comienza con un objeto principal llamado **document**.

- document representa **toda la página web**
- Es el punto de entrada para acceder a cualquier elemento HTML.
- Sin document, JavaScript no sabría dónde buscar.

Piensa en document como el **mapa completo de la página**.

Nota: Todo lo que quieras hacer en una página web empieza desde document.

2. Los elementos HTML

Cada etiqueta HTML se convierte en un **elemento del DOM**.

Ejemplos de elementos:

- `<h1>` → encabezado
- `<p>` → párrafo
- `<button>` → botón
- `<input>` → campo de texto

Estos elementos pueden:

- Tener contenido.
- Tener atributos.
- Cambiar su comportamiento.

Nota: Cada etiqueta HTML es un objeto que puede ser manipulado.



3. Los nodos

En el DOM, todo se considera un **nodo**. Existen varios tipos, pero los más importantes son:

- **Nodo elemento:** etiquetas HTML (div, p, button)
- **Nodo texto:** el texto dentro de una etiqueta
- **Nodo atributo:** propiedades como id, class, src

Ejemplo:

```
<p id="mensaje">Hola</p>
```

- <p> → nodo elemento
- id="mensaje" → nodo atributo
- Hola → nodo texto

Nota: El DOM no solo ve etiquetas, también ve textos y atributos.

4. Relación entre nodos

Los nodos se relacionan entre sí como una familia:

- **Padre:** el nodo que contiene a otros.
- **Hijo:** el nodo que está dentro de otro.
- **Hermano:** nodos que comparten el mismo padre.

Ejemplo:

```
<div>  
  <h2>Título</h2>  
  <p>Descripción</p>  
</div>
```

- div → nodo padre
- h2 y p → nodos hijos
- h2 y p → nodos hermanos

Nota: Entender estas relaciones te permite moverte dentro del DOM.



¿Cómo accede JavaScript al DOM?

Para que una página web sea interactiva, JavaScript necesita **encontrar** los elementos HTML dentro del DOM. Para lograrlo, se utilizan **métodos de selección**.

Piensa en esto como si le dieras instrucciones a JavaScript para decirle:

“Busca este elemento específico dentro de la página”

1. Seleccionar un elemento por su id

El id identifica de forma **única** a un elemento.

HTML

```
<p id="mensaje">Hola mundo</p>
```

JavaScript

```
document.getElementById("mensaje");
```

¿Qué hace este código?

Busca en el DOM el elemento cuyo id sea "mensaje".

Cuando usarlo:

Cuando necesitas acceder a un elemento específico y único.

2. Seleccionar elementos por su clase

Las clases permiten agrupar varios elementos.

HTML

```
<p class="texto">Texto 1</p>  
<p class="texto">Texto 2</p>
```



JavaScript

```
document.getElementsByClassName("texto");
```

Nota: Este método devuelve varios elementos, no uno solo.

3. Seleccionar elementos por su etiqueta

Permite acceder a todos los elementos de un mismo tipo.

JavaScript

```
document.getElementsByTagName("p");
```

Nota: Obtiene todos los párrafos de la página.

4. Selectores modernos (como en CSS)

JavaScript también puede usar selectores similares a CSS.

JavaScript

```
document.querySelector("#mensaje");  
document.querySelector(".texto");  
document.querySelector("button");
```

Ventaja: Es más flexible y fácil de leer.

Interacción con el DOM: los eventos

Hasta este punto has aprendido que JavaScript puede **acceder** a los elementos de una página web y **modificar** su contenido. Sin embargo, para que una página sea realmente interactiva, es necesario que **reaccione a las acciones del usuario**. Estas acciones se conocen como **eventos**.



¿Qué es un evento?

Un **evento** es cualquier acción que realiza el usuario sobre una página web, por ejemplo:

- Hacer clic en un botón.
- Escribir en un campo de texto.
- Enviar un formulario.
- Mover el mouse.
- Presionar una tecla.

El DOM permite detectar estos eventos y ejecutar una acción como respuesta.

Nota: Un evento es la forma en que la página “escucha” al usuario.

Eventos comunes en el desarrollo web

Algunos de los eventos más utilizados son:

- click → cuando el usuario hace clic.
- input → cuando el usuario escribe en un campo.
- submit → cuando se envía un formulario.
- mouseover → cuando el mouse pasa sobre un elemento.

Estos eventos permiten crear experiencias dinámicas y mejorar la interacción con la interfaz.

Cómo se relacionan los eventos con el DOM

Cada elemento del DOM puede **escuchar eventos**. Cuando ocurre el evento, JavaScript ejecuta una función.

Ejemplo conceptual:

- El usuario hace clic en un botón.
- El botón detecta el evento.
- JavaScript ejecuta una acción.



- El DOM se actualiza.

Todo este proceso ocurre **sin recargar la página**.

Ejemplo conceptual de interacción

Imagina el siguiente escenario:

- Un botón con el texto *“Mostrar mensaje”*.
- Un párrafo vacío.

Cuando el usuario hace clic en el botón:

- El párrafo aparece.
- El contenido se actualiza.
- La página responde de inmediato.

Este comportamiento es posible porque el DOM permite **escuchar eventos y modificar elementos en tiempo real**.

Relación entre DOM, JavaScript y eventos

Para que exista interacción, siempre se combinan tres elementos:

1. **El DOM**: representa la estructura de la página.
2. **JavaScript**: contiene la lógica.
3. **Los eventos**: conectan al usuario con la página.

Nota: El DOM permite que JavaScript reaccione a las acciones del usuario.

Creación de elementos con el DOM

Hasta ahora has aprendido que el DOM permite **acceder** a los elementos de una página y **responder a eventos** como clics o escritura. El siguiente paso es entender que el DOM no solo modifica lo que ya existe, sino que también permite **crear nuevos elementos** dinámicamente.



Esto significa que una página web puede **crecer y cambiar en tiempo real** según las acciones del usuario.

¿Por qué crear elementos con el DOM?

Imagina los siguientes escenarios:

- Agregar tareas a una lista sin recargar la página.
- Mostrar mensajes nuevos cuando el usuario interactúa.
- Crear tarjetas, alertas o filas en una tabla dinámicamente.
- Construir interfaces que se adapten al usuario.

Todo esto es posible gracias a la **creación de elementos con el DOM**.

Nota: El DOM permite crear, modificar y eliminar elementos mientras la página está en uso.

Cómo se crea un elemento en el DOM

Para crear un nuevo elemento, JavaScript sigue un proceso sencillo:

- **Crear el elemento:** Se indica qué tipo de etiqueta se va a crear (div, p, li, etc.).
- **Agregar contenido o atributos:** Se define el texto, clases o identificadores del elemento.
- **Insertar el elemento en la página:** Se añade el nuevo elemento dentro de otro elemento existente.

Este proceso se realiza completamente a través del DOM.

Ejemplo conceptual

Imagina una página que tiene un botón llamado **“Agregar mensaje”**.

Cuando el usuario hace clic:

- Se crea un nuevo párrafo.



- Se agrega un texto.
- El párrafo aparece en la pantalla.

Este flujo muestra cómo un **evento** puede provocar la **creación de nuevos elementos**, haciendo que la página responda de forma dinámica.

Relación entre eventos y creación de elementos

La creación de elementos casi siempre está asociada a un evento:

- El usuario hace clic.
- JavaScript detecta el evento.
- Se crea un nuevo elemento.
- El DOM se actualiza.

Nota: Los eventos son el detonante; el DOM es el medio para crear y mostrar nuevos elementos.

JavaScript

```
// Crear un nuevo párrafo
const parrafo = document.createElement("p");

// Agregar contenido
parrafo.textContent = "Este párrafo fue creado con el DOM";

// Insertar en la página
document.body.appendChild(parrafo);
```

Materiales de formación:

Visual estudio code

Computador



Navegador

Estrategias didácticas activas:

- Clase magistral breve con mediación activa.
- Aprendizaje activo
- Aprendizaje significativo

Duración de la actividad 1 hora.

C. Actividades de apropiación del conocimiento.

En los momentos anteriores has comprendido qué es el DOM, cómo el navegador organiza una página web y de qué manera JavaScript puede interactuar con sus elementos para responder a eventos y crear contenido dinámico. Ahora es el momento de **poner ese conocimiento en acción** y hacerlo propio a través de la práctica guiada.

En este momento de apropiación comenzarás a **utilizar el DOM de forma consciente**, entendiendo no solo qué instrucciones escribir, sino **por qué se usan y qué efecto producen** sobre la estructura de la página. Cada acción que realices te permitirá fortalecer la relación entre la teoría y su aplicación en situaciones reales del desarrollo web.

El objetivo de este momento es que desarrolles seguridad en el uso del DOM, reconociendo cómo seleccionar elementos, modificar su contenido, responder a eventos y crear nuevos elementos de manera estructurada. Este proceso te ayudará a construir una base sólida para enfrentar retos más complejos en el desarrollo de aplicaciones web interactivas.

A partir de aquí, el aprendizaje se vuelve más práctico y progresivo, permitiéndote consolidar las competencias necesarias para transformar una página web estática en una interfaz dinámica y funcional.



Actividades Prácticas:

Propósito de la actividad

Aplicar de manera práctica los conceptos del DOM para seleccionar elementos, responder a eventos y crear nuevos elementos dinámicamente, comprendiendo cómo cambia la estructura de la página web en tiempo real.

Contexto

Vas a trabajar sobre una página web sencilla que contiene algunos elementos básicos. A partir de la interacción del usuario, la página deberá responder mostrando información nueva sin necesidad de recargarla.

Esta actividad te permitirá entender cómo el DOM actúa como el puente entre el HTML y JavaScript para crear interactividad.

Escenario de trabajo

Imagina que estás desarrollando una página web para registrar mensajes de un usuario. La página cuenta con:

- Un título
- Un campo de texto
- Un botón
- Un espacio donde se mostrarán los mensajes ingresados

Cada vez que el usuario interactúe con el botón, la página deberá **crear un nuevo elemento** y mostrarlo en pantalla.

Indicaciones generales



1. Observa la estructura HTML entregada por el instructor.
2. Identifica los elementos que necesitas manipular desde JavaScript.
3. Asocia un evento al botón.
4. Crea un nuevo elemento usando el DOM.
5. Agrega contenido al elemento creado.
6. Inserta el nuevo elemento dentro de la página.
7. Valida que los campos no estén vacíos o con espacios en blanco:
 - a. Si los campos estan vacíos, responde con un mensaje indicando que debe tener información.
 - b. Todos los campos deben estar llenos para poder realizar el envío del formulario.

Durante el proceso, analiza cómo el DOM se va modificando con cada acción del usuario.

Puntos de reflexión durante la actividad

Mientras desarrollas la actividad, responde por escrito:

- ¿Qué elemento del DOM estás seleccionando?
- ¿Qué evento provoca el cambio en la página?
- ¿Qué nuevo elemento se crea?
- ¿Dónde se inserta ese elemento dentro del DOM?
- ¿Qué ocurre en la página cada vez que repites la acción?

Resultado esperado

Al finalizar la actividad, deberás observar que:

- La página responde a una acción del usuario.
- Se crean nuevos elementos sin recargar la página.
- La estructura del DOM cambia dinámicamente.
- JavaScript controla el comportamiento de la interfaz.



Materiales de formación:

- Planteamiento de los ejercicios
- Pc
- Navegador
- Visual estudio code
- Página inicial para el ejercicio

Estrategias didácticas activas:

- Acompañamiento individual
- Resolución progresiva
- Aprendizaje por descubrimiento
- Debate técnico breve
- Retroalimentación continua

Evidencias de aprendizaje:

- Código fuente con repositorio

Duración de la actividad 2 hora.

D. Actividades de transferencia del conocimiento.

Hasta este punto has aprendido a utilizar el DOM para interactuar con los elementos de una página web, responder a eventos y crear contenido dinámico mediante JavaScript. Ahora es el momento de **llevar este conocimiento a situaciones más cercanas a la realidad del entorno laboral**, donde los problemas no se presentan de forma aislada, sino como parte de una solución completa.

En este momento aplicarás lo aprendido para resolver una necesidad concreta, tomando decisiones sobre cómo estructurar el código, qué elementos manipular y de qué manera



responder a las acciones del usuario. El objetivo no es solo que el resultado funcione, sino que comprendas **por qué eliges una solución y cómo el DOM te ayuda a implementarla.**

La transferencia del conocimiento te permitirá demostrar que eres capaz de usar el DOM como una herramienta para construir funcionalidades reales, fortaleciendo tu autonomía, tu pensamiento lógico y tu capacidad para enfrentar retos propios del desarrollo web. Este proceso te acerca a escenarios profesionales donde la interacción, la usabilidad y la dinámica de la interfaz son fundamentales.

Actividades de aprendizaje:

En un entorno real de desarrollo web, es común trabajar con sistemas que deben **consultar información existente** y, a partir de ella, **registrar nuevos datos sin recargar la página**, garantizando una experiencia fluida para el usuario.

Para esta actividad, trabajarás con el servidor de pruebas previamente utilizado en clase:

<https://jsonplaceholder.typicode.com>

Este servicio simula una base de datos y será utilizado para **consultar usuarios existentes** y **registrar tareas asociadas a cada uno.**

Objetivo de la actividad

Aplicar los conocimientos del **DOM**, el manejo de **eventos**, la **creación dinámica de elementos** y el uso de **formularios sin recarga**, mediante la construcción de un módulo básico de gestión de tareas por usuario

Descripción general del ejercicio



El sistema debe permitir **buscar un usuario existente** y, una vez encontrado, **registrar tareas asociadas a dicho usuario**. Todo el proceso debe realizarse **sin recargar la página**, utilizando eventos y manipulación del DOM.

Es importante tener en cuenta que:

- Los usuarios **ya existen en la base de datos**.
- No se deben crear nuevos usuarios.
- El enfoque principal es el **manejo del DOM y la interacción dinámica**.

Análisis del problema

Antes de iniciar el desarrollo, analiza cuidadosamente el enunciado e identifica:

- ¿Qué información debe ingresar el usuario?
- ¿Qué acciones debe realizar el sistema?
- ¿Qué resultados debe mostrar la interfaz?

Datos de entrada

El sistema debe recibir como entrada:

1. Documento del usuario:
 - Ingresado mediante una **caja de texto**.
 - Utilizado para buscar el usuario en el servidor.
2. **Datos de la tarea**, tales como:
 - Título de la tarea.
 - Descripción.
 - Estado de la tarea.
 - Cualquier otro campo definido por el instructor.

Procesos a realizar



El sistema debe ejecutar los siguientes procesos:

1. **Búsqueda del usuario**

- Capturar el evento de envío del formulario sin recargar la página.
- Consultar el servidor para verificar si el usuario existe.
- Mostrar en pantalla los datos del usuario encontrado en campos previamente definidos.
- Si el usuario no existe, mostrar un mensaje indicando que no está registrado en el sistema.

2. **Registro de tareas**

- Habilitar el formulario de registro de tareas únicamente cuando el usuario haya sido encontrado.
- Validar que todos los campos del formulario de tareas estén completos.
- Enviar la información de la tarea sin recargar la página.
- Asociar la tarea al usuario correspondiente.

3. **Manipulación del DOM**

- Crear dinámicamente los elementos necesarios para mostrar la información.
- Agregar cada nueva tarea a una tabla visible en la interfaz.
- Mostrar las tareas registradas una debajo de la otra.

Datos de salida

El sistema debe mostrar como resultado:

- Los **datos del usuario** consultado.
- Un **mensaje informativo** cuando el usuario no sea encontrado.
- Una **tabla dinámica** con las tareas registradas para el usuario.
- La actualización de la interfaz en tiempo real, sin recargar la página.

Condición de finalización



La actividad se considera finalizada cuando:

- Se ha buscado correctamente un usuario existente.
- Se han registrado una o más tareas asociadas a ese usuario.
- Las tareas se visualizan correctamente en la tabla.
- Todo el proceso se realiza mediante el uso adecuado del DOM y eventos.

Criterio clave de evaluación

Durante el desarrollo de esta actividad se evaluará principalmente:

- El uso correcto del DOM para seleccionar, crear y modificar elementos.
- La gestión de eventos en formularios.
- La manipulación dinámica de la interfaz sin recargar la página.
- La capacidad de análisis y comprensión del problema planteado.

Duración de la actividad 5 horas.

1. ACTIVIDADES DE EVALUACIÓN

Tome como referencia la técnica e instrumentos de evaluación citados en la guía de Desarrollo Curricular

Fase del proyecto formativo	Actividad del proyecto formativo	Actividad de Aprendizaje	Evidencias de Aprendizaje	Criterios de Evaluación	Técnicas e Instrumentos de Evaluación
EJECUCIÓN	DESARROLLAR LA ESTRUCTURA DE DATOS, CONSTRUCCIÓN, CODIFICACIÓN E INTERFAZ DE USUARIO, PRUEBAS Y	Identifica y describe, en un sistema de información dado, los datos de entrada, procesamiento de los datos	Conocimiento y Producto	Solucionar problemas de lógica proposicional. Usar estructuras secuenciales	Listas de chequeo



	VALIDACIÓN DE LOS MÓDULOS DEL SISTEMA DE INFORMACIÓ	e información generada, según necesidades del cliente.		en la construcción de algoritmos. Utilizar estructuras de control en la construcción de algoritmos. Emplear estructuras cíclicas en la construcción de algoritmos. Construir algoritmos con arreglos.	
--	--	--	--	--	--

2. GLOSARIO DE TÉRMINOS

DOM (Document Object Model): Modelo que representa la estructura de un documento HTML como un conjunto de objetos organizados en forma de árbol, permitiendo que JavaScript acceda, modifique y cree elementos de una página web.

Documento (document): Objeto principal del DOM que representa toda la página web. Es el punto de partida para acceder a cualquier elemento del documento.

Nodo: Unidad básica del DOM. Puede representar un elemento HTML, un texto o un atributo dentro de la estructura del documento.

Nodo padre: Elemento del DOM que contiene a otros nodos dentro de su estructura.

Nodo hijo: Elemento que se encuentra dentro de otro nodo y depende de él.



Nodo hermano: Nodos que comparten el mismo nodo padre dentro del DOM.

Elemento HTML: Objeto del DOM que representa una etiqueta HTML, como div, p, button, input, entre otras.

Evento: Acción realizada por el usuario o el navegador que puede ser detectada por JavaScript, como un clic, la escritura en un campo o el envío de un formulario.

Manejador de eventos: Función que se ejecuta cuando ocurre un evento específico sobre un elemento del DOM.

Formulario: Conjunto de campos que permite al usuario ingresar información y enviarla para su procesamiento.

Evento submit: Evento que se activa cuando un formulario es enviado. Permite capturar y controlar el envío de datos sin recargar la página.

Interactividad: Capacidad de una página web para responder a las acciones del usuario de forma dinámica.

Manipulación del DOM: Proceso mediante el cual JavaScript accede, modifica, elimina o crea elementos dentro de la estructura del documento.

Creación de elementos: Acción de generar nuevos elementos HTML dinámicamente utilizando el DOM, sin necesidad de escribirlos previamente en el archivo HTML.

Validación de formularios: Proceso que verifica que los datos ingresados por el usuario cumplan con los criterios establecidos antes de ser enviados.

JSON: Formato de intercambio de datos basado en texto, ampliamente utilizado para la comunicación entre aplicaciones web y servidores.



API: Conjunto de reglas y servicios que permiten la comunicación entre diferentes sistemas o aplicaciones.

JSONPlaceholder: Servicio de prueba que simula una API REST, utilizado para realizar prácticas de consultas y registros de datos sin afectar una base de datos real.

CRUD: Acrónimo de Create, Read, Update y Delete. Representa las operaciones básicas para la gestión de información en una aplicación.

Interfaz de usuario (UI): Parte visual de una aplicación web con la que interactúa el usuario.

Recarga de página: Proceso mediante el cual el navegador vuelve a cargar todo el contenido del sitio. En aplicaciones modernas se busca evitarla para mejorar la experiencia del usuario.

3. REFERENTES BIBLIOGRÁFICOS

Mozilla Developer Network (MDN). (s. f.). *Introducción al DOM*.

Mozilla Foundation.

Material de referencia oficial sobre el Modelo de Objetos del Documento, su estructura, nodos, eventos y manipulación con JavaScript.

Mozilla Developer Network (MDN). (s. f.). *Manipulación del DOM con JavaScript*.

Mozilla Foundation.

Documentación técnica para la selección, modificación y creación de elementos dinámicos en el DOM.

Flanagan, D. (2020). *JavaScript: The Definitive Guide* (7th ed.). O'Reilly Media.

Libro de referencia avanzado que aborda JavaScript y su interacción con el DOM en aplicaciones web modernas.

Duckett, J. (2014). *JavaScript and JQuery: Interactive Front-End Web Development*. Wiley.

Texto introductorio orientado al desarrollo frontend y a la creación de interfaces dinámicas.



W3C. (s. f.). *Document Object Model (DOM) Standard*.

World Wide Web Consortium.

Especificación oficial del estándar DOM utilizada por los navegadores web.

JSONPlaceholder. (s. f.). *Fake Online REST API for Testing and Prototyping*.

Servicio de simulación de API REST utilizado para prácticas de consumo de datos y pruebas de CRUD.

SENA. (2020). *Lineamientos pedagógicos para la formación profesional integral*.

Servicio Nacional de Aprendizaje – SENA.

Documento institucional que orienta la estructuración de guías de aprendizaje y momentos pedagógicos.

4. CONTROL DEL DOCUMENTO

	Nombre	Cargo	Dependencia	Fecha
Autor (es)	John Freddy Becerra Castellanos	Instructor Contratista	CIMI	03/02/2026

5. CONTROL DE CAMBIOS (diligenciar únicamente si realiza ajustes a la guía)

	Nombre	Cargo	Dependencia	Fecha	Razón del Cambio
Autor (es)					